Terraform vs Ansible

Which Tool Should You Use?

Infrastructure
vs Configuration

Many DevOps engineers
confuse these tools.

Let's break it down simply ⬇

Terraform

↓

Creates Cloud Infrastructure

↓

VM / Kubernetes / Network

DevOps Shack

Terraform vs Ansible

1. Introduction

Modern cloud infrastructure requires **automation, scalability, and repeatability**.
Two of the most widely used tools for infrastructure automation are:

- **Terraform**
- **Ansible**

Although they are often mentioned together in DevOps workflows, **they solve different problems**.

Understanding the difference is critical for designing **production-grade infrastructure automation pipelines**.

In simple terms:

Tool	Category	Purpose
Terraform	Infrastructure as Code (IaC)	Creates infrastructure
Ansible	Configuration Management	Configures infrastructure

A typical DevOps architecture uses **both together**.

Example pipeline:

Terraform → Creates infrastructure
Ansible → Configures servers
CI/CD → Deploys applications

This document explains:

- How both tools work
- Their architectural differences
- Real production use cases
- When to use each tool
- How they complement each other

---

2. Overview of Terraform

What is Terraform?

Terraform is an **Infrastructure as Code (IaC)** tool created by **HashiCorp**.

It allows engineers to **define infrastructure using code** and provision it automatically across multiple cloud providers.

Terraform uses **declarative configuration language (HCL)**.

Example:

```
resource "aws_instance" "web" {
  ami          = "ami-0abc123"
  instance_type = "t3.micro"
}
```

Running:

```
terraform apply
```

creates the infrastructure.

Terraform Architecture

Terraform works using:

Terraform CLI
↓
Terraform Configuration (HCL)
↓
Terraform Providers
↓
Cloud APIs
↓
Infrastructure Created

Example providers:

- AWS
- Azure
- Google Cloud
- Kubernetes
- GitHub
- Cloudflare

Terraform Core Concepts

1. Providers

Providers connect Terraform to APIs.

Example:

```
provider "aws" {
  region = "ap-south-1"
}
```

---

2. Resources

Resources represent infrastructure objects.

Examples:

aws_instance
aws_vpc
aws_subnet
aws_security_group

---

3. State File

Terraform maintains a **state file**.

terraform.tfstate

This file tracks:

Infrastructure created
Resource IDs
Dependencies
Changes

Without state, Terraform cannot track infrastructure.

---

4. Execution Plan

Terraform uses a **plan before execution**.

terraform plan

Example output:

+ aws_instance.web will be created

Then apply:

terraform apply

---

5. Dependency Graph

Terraform automatically calculates dependencies.

Example:

VPC
↓
Subnets
↓
EC2 Instances

Terraform builds infrastructure in the correct order.

---

3. Overview of Ansible

What is Ansible?

Ansible is a **configuration management and automation tool** developed by **Red Hat**.

It is used for:

- Server configuration
- Application deployment
- Automation
- Patch management
- Orchestration

Unlike Terraform, **Ansible does NOT create infrastructure directly.**

Instead it configures existing systems.

Ansible Architecture

Ansible Control Node
↓
SSH
↓
Managed Nodes
↓
Playbooks Execute Tasks

Key property:

Ansible is **agentless**.

It connects using:

SSH

No software required on target machines.

Ansible Core Concepts

Inventory

Defines target machines.

Example:

```
[web]
10.0.1.5
10.0.1.6
```

Playbooks

Playbooks define automation tasks.

Example:

```
- hosts: web
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
```

Modules

Ansible modules perform operations.

Examples:

apt
yum
copy
template
docker_container
service

Roles

Roles structure large automation projects.

Example structure:

```
roles/
  nginx/
    tasks/
    templates/
    handlers/
```

---

4. Key Differences Between Terraform and Ansible

Feature	Terraform	Ansible
Tool Type	Infrastructure as Code	Configuration Management
Language	HCL	YAML
Approach	Declarative	Procedural
Primary Purpose	Provision infrastructure	Configure systems
State Management	Maintains state file	No state tracking
Execution Model	Resource dependency graph	Sequential tasks
Agent Requirement	None	None

Feature	Terraform	Ansible
Communication	Cloud APIs	SSH
Multi-cloud support	Strong	Limited
Infrastructure lifecycle	Full lifecycle	Mostly configuration

---

5. Diagram Comparison

```
DEVOPS AUTOMATION PIPELINE

Developer
↓
Git Repository
↓
CI/CD Pipeline
↓
┌──────────────────────┐
│      TERRAFORM       │
│ Infrastructure as Code │
└──────────────────────┘
↓
Creates Infrastructure
┌──────────────┬──────────────┐
│   VPC        │   Network    │
│              │              │
│   EC2        │ LoadBalancer │
│              │              │
│ Kubernetes   │ SecurityGrp  │
└──────────────┴──────────────┘
↓
┌──────────────────────┐
│      ANSIBLE         │
│ Configuration Manager │
└──────────────────────┘
↓
Configures Infrastructure

Install Docker
Configure Nginx
Deploy Application
Setup Monitoring
↓
Application Ready
```

---

6. Real-World Terraform Use Cases

Example 1: Creating AWS Infrastructure

Terraform code creates:

- VPC
- Subnets
- EC2 instances
- Security groups
- Load balancer

Example:

Terraform
↓
AWS Cloud
↓
VPC
Subnets
EC2
Load Balancer

Example code:

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}
```

Example 2: Kubernetes Infrastructure

Terraform provisions:

- EKS cluster
- Node groups
- IAM roles
- Networking

Example:

terraform apply

creates:

Kubernetes Cluster
Worker Nodes
Networking
IAM Roles

Example 3: Multi-cloud Infrastructure

Terraform can deploy across:

AWS
Azure
Google Cloud

Example scenario:

Frontend → AWS
Backend → GCP
Database → Azure

Terraform orchestrates everything.

---

7. Real-World Ansible Use Cases

Example 1: Server Configuration

Ansible installs software on servers.

Example:

Install Docker
Install Nginx
Configure Firewall
Deploy application

Example playbook:

```
- hosts: web
  tasks:

    - name: Install Docker
      apt:
        name: docker.io
        state: present
```

Example 2: Application Deployment

Ansible deploys applications.

Example pipeline:

CI/CD builds container
↓
Push to registry
↓
Ansible deploys container

Example task:

docker_container

Example 3: Patch Management

Ansible can automate updates.

Example:

Update 100 servers
Apply security patches
Restart services

Example playbook:

```
- hosts: all
  tasks:
    - name: Update packages
      apt:
        upgrade: dist
```

---

8. Using Terraform and Ansible Together

In production environments, both tools are often combined.

Typical DevOps architecture:

Terraform
↓
Creates infrastructure

EC2 Instances
VPC
Subnets
Load Balancer

↓

Ansible
↓
Configures servers

Install Docker
Setup Nginx
Deploy App
Configure Monitoring

Example workflow:

terraform apply
↓
servers created
↓
ansible-playbook setup.yml

---

9. Common Misconceptions

Misconception 1

"Terraform replaces Ansible"

Incorrect.

Terraform provisions infrastructure.
Ansible configures it.

Misconception 2

"Ansible can replace Terraform"

Not fully.

While Ansible can create infrastructure using modules, it is **not designed for full infrastructure lifecycle management.**

Terraform handles:

- dependency graphs
- state management
- infrastructure lifecycle

much better.

Misconception 3

"Both tools compete"

They **complement each other**.

Most production pipelines use both.

---

10. When to Use Terraform

Use Terraform when you need to:

Create infrastructure automatically

Examples:

Cloud environments
Kubernetes clusters
Networking
Load balancers
Cloud storage

Best for:

Infrastructure provisioning
Cloud automation
Multi-cloud deployments

---

11. When to Use Ansible

Use Ansible when you need to:

Configure servers.

Examples:

Install Docker
Configure Nginx
Deploy applications
Manage patches

Best for:

Server configuration
Application deployment
Automation

---

12. Conclusion

Terraform and Ansible are **both critical tools in modern DevOps ecosystems**, but they serve different roles.

Terraform focuses on:

Infrastructure provisioning
Cloud automation
Multi-cloud deployment

Ansible focuses on:

Configuration management
Server automation
Application deployment

In production environments, the best practice is:

Terraform → Create infrastructure
Ansible → Configure infrastructure
CI/CD → Deploy applications

Using both tools together allows DevOps teams to build **fully automated, scalable, and repeatable infrastructure pipelines**.